

MACHINE UNIVERSELLE

Par Isaac GHORZI, Clémence GERMETTE et Louiza MAKHOUKHENE

Ce document détaille le développement d'un simulateur de machine de Turing, aboutissant vers la simulation de la machine universelle simulant elle-même d'autres machines. Un langage a été défini pour programmer les machines simulées par l'interpréteur. Pour simuler une machine via la machine universelle, un codage du langage mentionné précédemment a été mis au point.

Certaines ressources et figures ont été placés en annexe pour ne pas encombrer le corps principal du document. Cependant, tout n'est pas inclus dans le document.

Ce projet est composé de plusieurs modules écrits en python, de scripts utilitaires et de codes de machines de Turing.

SOMMAIRE

1. Simulation des machines de Turing	2
1.1. Définitions	2
1.2. Implémentation	2
1.3. Exemples de machines valides	2
2. Machine Universelle	4
2.1. Codage des éléments de la machine	4
2.2. Implémentation	4
2.3. Décidabilité (Question 11)	5
3. Annexe	6
3.1. Spécifications du langage	6
3.2. Code de LESS	7
3.3. Code de SEARCH	8
3.4. Code de UNARY_MULTIPLY	9

1. SIMULATION DES MACHINES DE TURING

1.1. Définitions

Le caractère blanc est noté $\square$ mais peut également être représenté par $_$ selon le contexte.

On note A notre alphabet d'entrée qui contient les lettres de la table ASCII.

On considère ici seulement les machines de Turing ayant pour alphabet de travail $\Gamma = \{0, 1, \#, |, \square\}$ et disposant de k rubans. De plus, ces machines n'effectuent seulement des calculs, c'est-à-dire qu'elles n'acceptent pas de mots, elles calculent simplement des sorties. On note I leur état initial et F leur état final.

1.2. Implémentation

Les machines de turing :

Les machines de Turing ont été implémenté sous forme d'une classe nommée `TuringMachine`. Elle est définie par un nom, sa table de transitions, et le nombre de ruban.

Une instance de cette classe, autrement dit une machine de Turing M , peut être instanciée à l'aide d'un fichier contenant du code de machine de Turing. Se référer à la Section 3.1 pour en savoir plus sur le langage utilisé.

Une fois que la machine M est initialisée correctement, il est possible de l'exécuter sur une entrée $w \in A$ à l'aide de sa méthode `run(entree: str)`. L'instance affiche à chaque étape de calcul sa configuration actuelle, permettant de suivre ce que fait la machine au fil du temps.

Les configurations :

Une configuration représente l'état d'une machine de Turing à un moment donné. Dans son implémentation, elle est définie par le nombre de ruban k , l'état actuel q et $\mathcal{R} = (r_1, r_2, \dots, r_k)$ la liste des rubans telle que $\forall i, 0 < i \leq k, r_i = (u, v)$ où u et v sont des piles.

La tête d'un ruban correspond au sommet de la pile v . Selon le déplacement de la tête :

- Pour aller à droite (R) on dépile le sommet de v et on l'empile dans u ;
- Pour aller à gauche (L), on dépile le sommet de u et on l'empile dans v ;
- Si on reste au même endroit (S) on ne fait rien.

Si l'une des deux piles est vide mais qu'on essaie d'y dépiler quelque chose, un $\square$ sera renvoyé à la place pour simuler le fait que le ruban soit infini dans les deux directions.

1.3. Exemples de machines valides

LESS : (code : Section 3.2)

Entrée : x, y (codage : $x\#y$)

Rubans : 2

- LESS déplace x vers le deuxième ruban;
- On ajoute des 0 à gauche du mot le plus petit pour qu'ils aient la même longueur;
- On compare chaque bits de gauche à droite :
 - Si le bit du ruban 1 et le bit du ruban 2 sont égaux, on continue;
 - Si on trouve 1 sur le ruban 1 et 0 sur le ruban 2, on s'arrête car ça signifie que $x < y$;
 - Sinon on boucle à l'infini;
- Si on arrive à la fin de chaque mot tel que $x = y$, on boucle à l'infini;

SEARCH : (code : Section 3.3)

Entrée : $x, w_1, w_2, \dots, w_l$ (codage : $x\#w_1\#w_2\# \dots \#w_l$)

Rubans : 2

- SEARCH déplace x vers le deuxième ruban;
- On compare x à chaque w de la liste bit à bit :
 - Si on repère une différence entre x et w on passe à l'élément suivant;
 - Si $x = w$ on s'arrête;
- Si aucun des membres de la liste ne correspond, on boucle à l'infini;

UNARY_MULTIPLY : (code : Section 3.4)

Entrée : $1^n \# 1^m$

Sortie : 1^{nm} sur le ruban 1 Rubans : 3

- La machine déplace la première chaîne de 1 vers le ruban 2 et supprime le #;
- La machine copie la deuxième chaîne de 1 vers le ruban 3;
- La machine copie une chaîne de 1^m dans le ruban 1 pour chaque 1 dans le ruban 2;

2. MACHINE UNIVERSELLE

2.1. Codage des éléments de la machine

Avant d'essayer de simuler une machine M à l'aide de la machine universelle U , on doit définir un codage pour représenter les éléments de M de manière à ce que U puisse les interpréter en raison de son alphabet de travail fini.

Codage des états :

Les états I et F sont respectivement notés 0 et 1. Pour les autres états, on passe par une fonction Num et Bin définies ainsi :

$$Num : Q \rightarrow \mathbb{N}, \quad Num(I) = 0, Num(F) = 1$$

$$Bin : \mathbb{N} \rightarrow \{0, 1\}^*$$

f associe à chaque état de Q un entier n , $n > 1$. Et g associe à tout entier sa représentation en binaire. Autrement dit :

$$\langle q \rangle = Bin(Num(q)), \forall q \in Q$$

Codage des transitions :

On définit le code suivant pour une transition tel que " $q|a|b|d|p$ " avec :

- $(q, p) \in Q$, l'état courant et le nouvel état;
- $a \in \{0, 1, <, -, >, \square\}$, le symbole sous la tête de lecture;
- $b \in \{0, 1, <, -, >, \square\}$, le symbole à écrire;
- $d \in \{<, -, >\}$, le déplacement de la tête de lecture;

Chaque transition est séparée par le symbole $|$.

"Que faudrait-il faire si on veut pouvoir accepter n'importe quel alphabet de travail ?"

Pour que M supporte n'importe quel alphabet de travail, il faudrait que la machine universelle U ne travaille qu'avec l'alphabet $\{0, 1\}$ et qu'un codage soit établi pour tout les symboles avec lesquels M puisse travailler. Un exemple concret de cette méthode est la table ASCII, qui associe à 256 lettres une suite unique de 8 bits.

2.2. Implémentation

Le code de la machine universelle avec et sans compteur est fourni avec le code source du projet, il n'a pas été inclus dans ce document pour sa longueur. On va expliquer ici le principe général de la machine.

On note U la machine universelle à 3 rubans. L'alphabet de travail de U , $\Gamma = \{0, 1, |, \#, \square, <, -, >\}$. La machine travaille avec 3 rubans dont le premier contient l'entrée $\langle M \rangle \#x$ au départ.

Initialisation :

C'est l'ensemble des états où U prépare la simulation de M . La machine va initialiser chacun de ses 3 rubans selon la configuration suivante :

1. $\# \langle M \rangle \#$, le code de M borné par deux $\#$;
2. $\langle q \rangle$, l'état courant de M ;
3. x , le mot donné en entrée à M ;

Recherche :

U va ensuite comparer q avec l'état de départ d'une transition dans le premier ruban, puis comparer le symbole sous la tête de lecture du ruban 3 avec le caractère de départ de la transition.

Si l'un des deux ne correspond pas, alors on saute cette transition et on répète les mêmes opérations sur la transition suivante. Si aucune transition ne correspond, alors U boucle à l'infini.

Cependant, s'il y a correspondance, alors la machine exécute la transition.

Transition :

La machine effectue les actions suivantes :

- Écrit le nouveau symbole sous la tête de lecture du ruban 3;
- Déplace la tête de lecture du ruban 3 selon le déplacement indiqué sur la transition;
- Le contenu du ruban 2 est effacé;
- On écrit le nouvel état dans le ruban 2;
- La tête de lecture du ruban 1 est déplacé à gauche du code de M;

À la fin, de la transition, la machine vérifie si on se trouve sur l'état final avant de soit poursuivre la simulation, soit y mettre fin.

Dans le contexte de la machine avec compteur, on décrémente le compteur unaire de 1 puis on regarde s'il ne reste plus rien avant de vérifier l'arrivée dans l'état final.

Après la réalisation de toutes ces vérifications, la machine se remet à chercher la prochaine transition à exécuter.

2.3. Décidabilité (Question 11)

Une machine de Turing Universelle à trois rubans est une machine qui est capable de simuler d'autres machines de Turing. Grâce à cette propriété, on peut alors mesurer le nombre d'étapes qu'il faut à une machine pour accepter, rejeter ou calculer l'image d'une entrée, ou bien limiter ce même nombre d'étape. Mais on peut surtout tester si une machine s'arrête pour une entrée donnée.

Cela nous permet d'évaluer la décidabilité d'un langage contenant des couples (machine, entrée). L'objet de cette section est de prouver la décidabilité d'un langage pour 3 cas distincts, cependant, on n'utilisera pas la simulation pour le prouver.

Les langages dont on va prouver la décidabilité sont les suivants :

1. $L_1 = \{ \langle M \rangle \# n \mid M \text{ s'arrête sur } n \text{ en moins de } n \text{ étapes} \}$
2. $L_2 = \{ \langle M \rangle \# n \mid M \text{ s'arrête sur les mots de taille } n \}$
3. $L_3 = \{ \langle M \rangle \# x \# y \mid M \text{ calcule la même chose sur les entrées } x \text{ et } y \}$

$L_1]$

Soit M_L la machine reconnaissant L_1 qui, sur une entrée n :

- Initialise un ruban avec un compteur selon n ;
- Exécute une machine M sur n :
 - Si M s'arrête sur n avant la fin du compteur, M_L accepte;
 - Si M ne s'arrête pas sur n avant la fin du compteur, M_L rejette;

Le temps de calcul est toujours borné par n . Donc L_1 est décidable quelque soit la machine et n .

$L_2]$

On suppose L_2 décidable par une machine M_2 . On construit H reconnaissant HALT, sur une entrée $\langle M \rangle \# w$:

- Sur n'importe quelle entrée x , on construit $\langle M_w \rangle$ ainsi :

1. Elle efface son entrée;
 2. Elle Simule M sur w ;
 3. Si la simulation s'arrête alors M_w s'arrête;
- On lance M_2 sur $\langle M_w \rangle \# 0$:
 - Si M_2 accepte alors H accepte;
 - Sinon, H rejette;

On a construit un décideur pour HALT à l'aide de M_2 . Or, HALT est indécidable par définition, donc c'est une contradiction. Donc L_2 est indécidable.

3. ANNEXE

3.1. Spécifications du langage

On définit dans cette section les spécifications du langage utilisé pour programmer des machines de Turing. Ces mêmes spécifications peuvent d'ailleurs être retrouvée dans le module `turingparser.py`

Un fichier est composé d'un en-tête suivi d'une liste de transitions. Les lignes vides sont ignorées. Les commentaires commencent par '#' et prennent la ligne entière. Cependant, un commentaire ne peut pas être écrit sur la même ligne qu'une transition.

Syntaxe :

```

<fichier>          ::= <en-tete> <transitions>

<en-tete>          ::= "name:" <string> "\n" "rubans:" <entier> "\n"

<commentaire>      ::= "#" <string>

<transitions>      ::= <transition> | <transition> "\n" <transitions>
<transition>       ::= <transition_debut> ";" <transition_fin>

<transition_debut> ::= <etat> "," <symboles_liste>
<transition_fin>  ::= <etat> "," <symboles_liste> "," <deplacements>

<etat>             ::= <string>

<symbole_liste>    ::= <symbole> | <sumbole> <symbole_liste>
<symbole>          ::= <char> | "_"

<deplacements>     ::= <direction> | <direction> <deplacements>
<direction>        ::= "<" | "L" | "-" | "S" | ">" | "R"

```

- <string> est une chaîne de caractères ordinaire.
- <char> est un caractère de la table ASCII.

3.2. Code de LESS

name: less

rubans: 2

entrée : x#y

si x et y n'ont pas la même taille

alors on ajoute des 0 à gauche du nombre le plus petit

e.g : 1#0010 => 0001#0010

si rien n'est donné en entrée, on ne fait rien

I,__;LOOP,__,--

on déplace x dans r2

I,1_;I,_1,>>

I,0_;I,_0,>>

on se place à droite de x et de y

I,#_;qRIGHT(1),__,>-

qRIGHT(1),0_;qRIGHT(1),0_,>-

qRIGHT(1),1_;qRIGHT(1),1_,>-

on ajoute des 0 à gauche du plus petit mot pour avoir une comparaison précise

qRIGHT(1),__,qSETUP,__,<<

qSETUP,00;qSETUP,00,<<

qSETUP,01;qSETUP,01,<<

qSETUP,10;qSETUP,10,<<

qSETUP,11;qSETUP,11,<<

qSETUP,0_;qSETUP,00,<<

qSETUP,_0;qSETUP,00,<<

qSETUP,1_;qSETUP,10,<<

qSETUP,_1;qSETUP,01,<<

on commence à comparer x et y

qSETUP,__,qTEST,__,>>

qTEST,11;qTEST,11,>>

qTEST,00;qTEST,00,>>

si x < y : on s'arrête

qTEST,10;F,10,--

sinon : on ne s'arrête pas

qTEST,__,qLOOP,__,--

qLOOP,__,qLOOP,__,--

3.3. Code de SEARCH

name: Recherche

rubans: 2

```
# entrée: x#w_1#w_2#...#w_l

# la machine va tester pour chaque w_i si x == w_i
# on s'arrête lorsqu'on trouve w_i : x == w_i

I,__;qLOOP,__,--

# on copie x dans r2
I,0_;I,_0,>>
I,1_;I,_1,>>

I,#_;qLEFT(2),#_,-<
qLEFT(2),#0;qLEFT(2),#0,-<
qLEFT(2),#1;qLEFT(2),#1,-<

qLEFT(2),#_;qTEST,#_,>>
qTEST,00;qTEST,00,>>
qTEST,11;qTEST,11,>>
qTEST,#_;F,#_,--
qTEST,__;F,__,--

qTEST,01;qSKIP,01,--
qTEST,10;qSKIP,10,--
qTEST,0_;qSKIP,0_,--
qTEST,1_;qSKIP,1_,--
qTEST,#1;qSKIP,#1,--
qTEST,#0;qSKIP,#0,--

qSKIP,0_;qSKIP,0_,>-
qSKIP,00;qSKIP,00,>-
qSKIP,01;qSKIP,01,>-
qSKIP,1_;qSKIP,1_,>-
qSKIP,10;qSKIP,10,>-
qSKIP,11;qSKIP,11,>-

qSKIP,__;qLOOP,__,--
qSKIP,_0;qLOOP,__,--
qSKIP,_1;qLOOP,__,--

qSKIP,#_;qLEFT(2),#_,--
qSKIP,#0;qLEFT(2),#0,--
qSKIP,#1;qLEFT(2),#1,--
```



```
qLOOP,__;qLOOP,__,--
```

3.4. Code de UNARY_MULTIPLY

```
name: Multiplication Unaire  
rubans: 3
```

```
# entrée : x#y
```

```
# sortie :  $x * y$  (sur r1)
```

```
# la machine va déplacer x dans r2, et copier y dans r3
```

```
# puis elle va copier le mot y x fois dans r1
```

```
I,__;F,__,---
```

```
I,1__;I,__1,>->
```

```
I,#__;qCOPY_y,__,>--
```

```
qCOPY_y,1__;qCOPY_y,11,>>-
```

```
qCOPY_y,__;qLEFT(2),__,-<-
```

```
# replace la tête de r2 à gauche
```

```
qLEFT(2),_1__;qLEFT(2),_1_,-<-
```

```
qLEFT(2),_11;qLEFT(2),_11,-<-
```

```
qLEFT(2),__1;qSTEP,__1,--<
```

```
# lorsqu'on commence, on décale la tête à gauche 2 fois
```

```
# pour ne pas multiplier par  $x + 1$ 
```

```
qLEFT(2),__;qBEGIN,__,--<
```

```
qBEGIN,__1;qADD,__1,-><
```

```
# on ajoute une copie de y dans r1
```

```
qADD,_11;qADD,111,>>-
```

```
qADD,__1;qLEFT(2),__1,-<-
```

```
qSTEP,__1;qADD,__1,->-
```

```
qSTEP,__;F,__,---
```